



Min-Max vs Z-Score Normalization in R: Feature Scaling Explained

Normalization Techniques

When attributes have different scales — such as salary in thousands and age in tens — algorithms tend to favor those with greater magnitude. Normalization solves this problem by transforming data to a common scale without distorting the relationships between them. Below, we explore three techniques, from the most widely used to the most didactic.

1

Min-Max (Scaling)

Compresses data into a fixed interval, typically [0, 1]. Formula: $v' = (v - \min) / (\max - \min)$

Ideal when the domain boundaries are known and preserving original proportions is essential — such as in neural networks and image processing.

2

Z-Score (Standardization)

Recenters data around mean 0 with standard deviation 1. Formula: $v' = (v - \mu) / \sigma$

Preferable when data follows an approximately normal distribution or when the presence of outliers needs to be maintained and interpreted with statistical rigor.

3

Decimal Normalization

Shifts the decimal point so that $\max(|v'|) < 1$. Formula: $v' = v / 10^j$, where j is the smallest integer satisfying this condition.

Conceptually simple, it serves as an introduction to the topic, but has limited use in modern data mining applications.

The choice between the two main techniques depends on the characteristics of the dataset:

When to Use Min-Max

- Domain boundaries are well known
- Data without severe outliers
- Applications with neural networks and images
- Need to preserve exact proportions

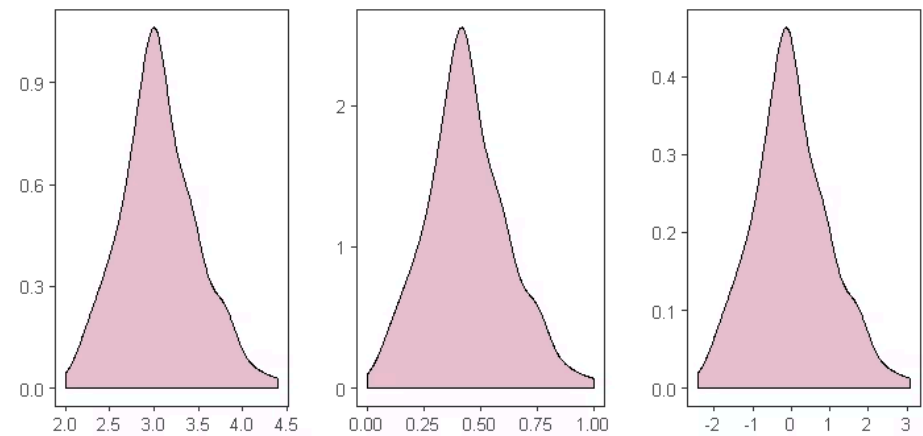
When to Use Z-Score

- Approximately normal distribution
- Presence of relevant outliers
- Domain boundaries are unknown
- Formal statistical analysis contexts

📖 **Reference:** E. Ogasawara et al., 2009, Neural networks cartridges for data mining on time series, Proceedings of the International Joint Conference on Neural Networks, p. 2302–2309.

Visual Comparison of Normalization

After understanding the techniques theoretically, it is time to observe their effect in practice. The chart below shows the same dataset subjected to each method — highlighting how the choice of technique changes the representation of values without losing the relational structure of the data.



Original Data

Starting point: raw values with natural scale and dispersion. Serves as a reference to compare the effect of each transformation applied below.

Min-Max [0-1]

All values are compressed into the interval [0, 1]. The shape of the distribution is preserved, but the extremes are anchored exactly at 0 and 1 — making outliers even more apparent.

Z-Score / N(0,1)

The data is recentered around zero with a standard deviation of 1. Unlike Min-Max, there are no fixed limits: outliers are retained and are interpreted in terms of how many deviations they are from the mean.

The comparison makes it clear that there is no universally superior technique — the best choice depends on the distribution of your data, the presence of outliers, and the requirements of the algorithm to be used.

Impact on the Model

Distance-sensitive algorithms (K-NN, SVM) can have their performance compromised without normalization — a single large-magnitude variable dominates the calculation.

Information Preservation

Neither technique discards data: they only rescale the values. The relative orderings and proportions between data points remain intact.

Practical Application

When deploying a model to production, save the parameters (min/max or μ/σ) calculated during training to apply the same transformation to new data.

Min-Max Normalization – Theory

Min-Max normalization is a **linear transformation** that maps every value in a dataset to a fixed interval — typically $[0, 1]$ — or any target range $[a, b]$. Because it is a monotonic linear rescaling, the **shape of the original distribution is fully preserved**: relative distances and orderings between values remain intact after transformation.

Formula

Standard mapping to $[0, 1]$:

$$v' = (v - \min) / (\max - \min)$$

- **v** — original value to be transformed
- **min** — minimum value of the training set
- **max** — maximum value of the training set
- **v'** — normalized output in $[0, 1]$

To map to an arbitrary interval $[a, b]$:

$$v' = a + (v - \min)(b - a) / (\max - \min)$$

Key Properties

- **Bounded output** — result is always within $[0, 1]$ for training data
- **Distance preservation** — relative distances between values are maintained after scaling
- **Outlier sensitivity** — a single extreme value compresses all other values into a narrow range, distorting the effective scale
- **Domain dependency** — requires knowing the minimum and maximum boundaries in advance; unseen values outside the training range produce outputs outside $[0, 1]$

✓ When to Use

- Known and stable domain boundaries (e.g., pixel values 0–255)
- Data free from severe outliers
- Neural networks with bounded activation functions (e.g., sigmoid, tanh)
- Image processing pipelines
- Algorithms requiring inputs within a fixed range

✗ When to Avoid

- Data with significant outliers — they will compress the majority of values
- Unknown or unbounded domain — min/max cannot be reliably estimated
- When statistical interpretability of deviations from the mean is required (prefer Z-Score instead)

⚠ **Data Leakage:** The values of **min** and **max** must be computed **exclusively on the training set** and then applied unchanged to the test set. Using test data to compute these parameters contaminates the model and invalidates evaluation results.

📖 **Reference:** Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann. Chapter 3 — Data Preprocessing, pp. 113–116.

Min-Max Normalization – minmax

With the overview consolidated, we move on to the practical implementation: see how to apply the `minmax` function from the **daltoolbox** package in R, from fitting to inverse transformation.

```
library(daltoolbox)
iris <- datasets::iris
# Sepal.Length: Min=4.3, Max=7.9

# Fit and transform
norm <- minmax()
norm <- fit(norm, iris)
ndata <- transform(norm, iris)
```

```
summary(ndata)
# Sepal.Length: Min=0.000, Max=1.000
# Sepal.Width:  Min=0.000, Max=1.000
# Petal.Length: Min=0.000, Max=1.000
# Petal.Width:  Min=0.000, Max=1.000

# Inverse transformation (verification)
ddata <- inverse_transform(norm, ndata)
summary(ddata)
# Sepal.Length: Min=4.3, Max=7.9
# (original values recovered)
```

Step by step

- **1. Instantiate** the normalizer with `minmax()` — it does not yet know the data
- **2. Fit** with `fit(norm, iris)` — captures the min and max of each numeric column
- **3. Transform** with `transform()`, which applies $(x - \text{min}) / (\text{max} - \text{min})$, mapping all values to `[0, 1]`
- **4. Verify** that the categorical column `Species` remains intact — **daltoolbox** automatically ignores non-numeric columns
- **5. Revert** with `inverse_transform()` to recover the original values and validate the consistency of the transformation

⚠ **Watch out for data leakage:** in the example above, `fit()` was called on the full dataset for didactic purposes only. In a real workflow, **fit the normalizer on the training set only** and apply it to the test set — using the test data to compute the min and max contaminates the model and invalidates the entire evaluation.

Z-Score Standardization – Theory

Z-Score standardization (also called z-normalization or standardization) is a transformation that recenters each variable around zero and scales it by its standard deviation. Unlike Min-Max, it does not impose fixed bounds — the output range depends on the data distribution. It is the method of choice when the data approximately follows a normal distribution or when outliers must be preserved and interpreted in a statistically meaningful way.

Formula

$$v' = (v - \mu) / \sigma$$

- **v** — original value to be transformed
- **μ** (mu) — mean of the training set
- **σ** (sigma) — standard deviation of the training set
- **v'** — standardized output

Customized target:

$$v' = \text{nmean} + (v - \mu) / \sigma \times \text{nsd}, \text{ allowing rescaling to a target mean (nmean) and standard deviation (nsd).}$$

✓ When to Use

- Approximately normal distribution
- Presence of relevant outliers that must be preserved
- Unknown domain boundaries
- Algorithms sensitive to scale but not to bounds (SVM with RBF kernel, PCA, k-NN, logistic regression)

Key Properties

- **Output centered at 0 with standard deviation 1** (for standard z-score)
- **No fixed bounds** — outliers are retained and expressed as multiples of σ
- **Robust to outliers** compared to Min-Max
- **Columns with zero variance** ($\sigma = 0$) cannot be standardized — they are typically mapped to the target mean (nmean)

✗ When to Avoid

- Algorithms requiring strictly bounded inputs (e.g., sigmoid activation in neural networks)
- When exact proportions relative to a fixed range must be preserved
- When the distribution is highly non-normal and outliers should be suppressed

⚠ **Data Leakage:** μ and σ must be computed **exclusively on the training set** and then applied to the test set. Using test data to compute these parameters contaminates the model.

Z-Score Standardization – zscore

While Min-Max compresses data into a fixed interval, Z-Score takes a different approach: it centers each column around zero and scales it by the standard deviation, without imposing upper or lower bounds. See how to apply the `zscore` function from the **daltoolbox** package in R.

```
library(daltoolbox)

iris <- datasets::iris

# Standard Z-Score (mean=0, sd=1)
norm <- zscore()
norm <- fit(norm, iris)
ndata <- transform(norm, iris)
summary(ndata)
# Sepal.Length: Mean=0.000, SD≈1
# Min=-1.864, Max=2.484
```

```
# Inverse transformation
ddata <- inverse_transform(norm, ndata)
# Recovers original values

# Z-Score with custom target
norm2 <- zscore(nmean=0.5,
               nsd=0.5/2.698)
norm2 <- fit(norm2, iris)
ndata2 <- transform(norm2, iris)
summary(ndata2)
# Sepal.Length: Mean=0.500
# Min≈0.155, Max≈0.960
```

Step by step

- **1. Instantiate** the normalizer with `zscore()` — without arguments, it will use mean 0 and standard deviation 1 as the target
- **2. Fit** with `fit(norm, iris)` — captures the mean (μ) and standard deviation (σ) of each numeric column
- **3. Transform** with `transform()`, which applies $(x - \mu) / \sigma$, centering the data around zero with unit variance
- **4. Customize** if needed: use `zscore(nmean, nsd)` to rescale to a different target mean and standard deviation — useful for compatibility with certain pipelines
- **5. Revert** with `inverse_transform()` to recover the original values and validate the consistency of the transformation

📌 **Contrast with Min-Max:** unlike Min-Max, Z-Score **does not limit the data range** — outliers still exist, but on a standardized scale. This makes it more robust for algorithms such as SVM with RBF kernel, PCA, and k-NN. **Special case:** columns with zero variance (constants) cannot be divided by $\sigma=0$; in these cases, **daltoolbox** maps them directly to the value `nmean`.



Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

